

Operators and Type Conversion

CSCI 1380 INTRODUCTION TO PROGRAMMING IN PYTHON



Topics

- ❑ Arithmetic Operators
- ❑ Assignment Operators
- ❑ Comparison Operators
- ❑ Logical/Boolean Operators
- ❑ Membership Operators
- ❑ Concatenation
- ❑ Type Conversion

Operators

Operators

Python has special symbols called **operators** to perform operations on variables and literals.

For example:

- `print( 5 + 7 ) # 11`

These are the type are the various types of operators.

- **Arithmetic Operators**
- **Assignment Operators**
- **Comparison Operators**
- **Logical/Boolean Operators**

Operator	Operation	Example
+	Addition	5 + 2 # gives us 7
-	Subtraction	4 - 2 # gives us 2
*	Multiplication	2 * 3 # gives us 6
/	Division	4 / 2 # gives us 2
//	Floor Division	10 // 3 # gives us 3
%	Modulo	5 % 2 # gives us 1
**	Power	4 ** 2 # gives us 16

Arithmetic Operators

Arithmetic operators are used to perform mathematical operations and follow the **PEMDAS** order of operations.

- **P**arenthesis
- **E**xponents (Power)
- **M**ultiplication
- **A**ddition
- **S**ubtraction

Assignment Operators

Assignment operators are used to **assign values to variables only** and can perform single mathematical operations.

Operator	Name	Example
=	Assignment Operator	a = 7
+=	Addition Assignment	a += 1 # a = a + 1
-=	Subtraction Assignment	a -= 3 # a = a - 3
*=	Multiplication Assignment	a *= 4 # a = a * 4
/=	Division Assignment	a /= 3 # a = a / 3
%=	Remainder Assignment	a %= 10 # a = a % 10
**=	Exponent Assignment	a **= 10 # a = a ** 10

Operator	Meaning	Example
==	Is Equal To	3 == 5 # gives us False
!=	Not Equal To	3 != 5 # gives us True
>	Greater Than	3 > 5 # gives us False
<	Less Than	3 < 5 # gives us True
>=	Greater Than or Equal To	3 >= 5 # give us False
<=	Less Than or Equal To	3 <= 5 # gives us True

Comparison Operators

Comparison operators compare two values or variables and **return a Boolean literal** of True or False.

Comparison Operators

It's important to note that comparison operators **are case sensitive and data type sensitive**. One exemption is True and False Boolean literals can be interchanged with 1 and 0 numerical literals.

For example:

- `"Hello" == "hello"` # gives us False, since it's case sensitive.
- `"7" == 7` # gives us False, since it's data type sensitive.
- `True == 1` # gives us True, since True and False can be represented as 1 and 0.

Comparison Operators (Strings)

Can Python compare if one String is larger than another?

Yes. Python compares String literals by using the **ASCII Table**. The ASCII Table assigns a numerical value to each character in a String. This is why comparison operators are case sensitive; a capital letter has a different numerical value to a lower-case letter.

When comparing greater or less than, Python starts by comparing the left-most character. If the left-most characters have the same numerical value, then the character to the right is compared next and so on. The length or size of a String is NOT considered.

- <https://media.geeksforgeeks.org/wp-content/uploads/20240304094301/ASCII-Table.png>

Operator	Example	Meaning
and	a and b	Logical AND: True only if both the operands are True
or	a or b	Logical OR: True if at least one of the operands is True
not	not a	Logical NOT: True if the operand is False and vice-versa.

Logical/Boolean Operators

Logical or Boolean Operators are used to **check if an expression is True or False** and return a Boolean value accordingly.

An expression can be an operation returning a Boolean value, a variable storing a Boolean value, or a Boolean literal.

For the examples, assume **a** and **b** are variables storing Boolean literal values.

Logical/Boolean Operators

The following table shows the output for all the possible combination of True and False expressions for logical/Boolean operators.

For the table bellow, **a** and **b** are variables.

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	
False	True	False	True	True
False	False	False	False	

Membership Operators

Operator	Meaning	Example
in	True if value/variable is found in the sequence	'll' in 'Hello' # gives us True
not in	True if value/variable is not found in the sequence	'x' not in 'Bye' # gives us True

Membership operators **check if a value is found in a sequence** and return True or False if the value is found.

A sequence can be a String, since it's a string or sequence of characters. A sequence can also be a List or Dictionary, which might be covered later throughout the course.

Concatenation

Concatenation

The + arithmetic operator allows us to add numerical values together, however it can also let us **merge Strings together**.

The process of merging Strings together is known as concatenation.

For example:

- `print( "Hello " + "World!" )` # gives us "Hello World!"
- `a = "foo"`
- `a += "bar"` # a now stores "foobar"

The * arithmetic operator allows us to multiply numerical values, however it also let's us merge Strings together x number of times.

For example:

- `print( "hip!" * 3 )` # gives us "hip!hip!hip!"

Concatenation

Python has an automatic concatenation feature whenever Strings are placed next to each other. This feature comes in handy when printing or assigning a long String value.

For example:

- `a = "Happy" "New" "Year" # assigns "HappyNewYear"`
- `print( "she sells sea shells"`
- `" by the sea shore" ) # outputs "she sells sea shells by the sea shore"`

Type Conversion

Type Conversion

As mentioned in the previous PowerPoint, **some data types don't interact well with others**. For example, the + arithmetic operator can be used to either add numerical values together or merge String together, but it can **NOT** add numerical values and Strings together.

For example:

- `print( "5" + 6 )` # this will throw an **ERROR**
- `print( "20" > 10 )` # this will throw an **ERROR**
- `print( "False" < True )` # this will throw an **ERROR**

Type Conversion

Python has built in functions that allows us to resolve these errors. These functions are known as **type conversion functions**. Type conversion functions receive a variable or literal as their argument and return a new literal in the desired data type format.

Type conversion functions:

- **str()** → converts other data types into a String
- **int()** → converts other data types into an Integer (whole number)
- **float()** → converts other data types into a Float (real number)
- **bool()** → converts other data types into a Boolean

- **type()** → let's us know the data type of a literal or variable

Type Conversion

We can use type conversion functions to resolve the errors thrown in the previous examples.

For example:

- `print( int("5") + 6 )` # 5 + 6 outputs **11**
- `print( "20" > str(10) )` # "20" > "10" outputs **True**
- `print( bool("False") < True )` # False < True outputs **True**

It's important to read the error messages shown in the terminal to know which variables or literals need to be converted.

Type Conversion

Note: Type conversion functions don't change the data type of a variable unless the assignment operator is used. Remember, the type conversion functions only receive an argument and return a literal value. They don't have the ability to modify the argument itself.

Example:

- `a = "5"`
- `int( a )`
- `print( type(a) )` # a is still type String
- `a = int( a )`
- `print( type(a) )` # a is now type Integer, since we used the = assignment operator.